

Instructions

SI and SS formats

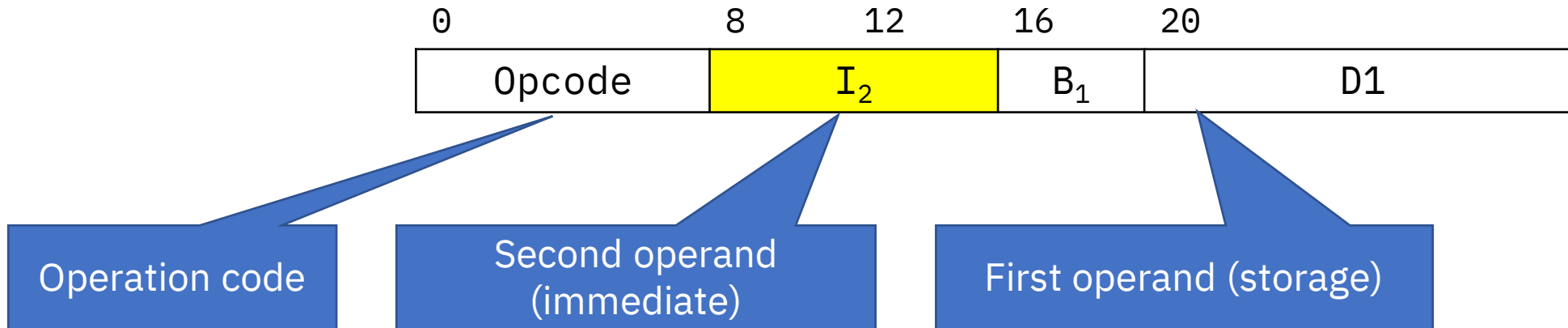
Instruction formats

- Next, SI instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register
Register-and-indexed-storage (RX)	and another a storage address
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address

SI instructions

- SI instructions occupy 2 halfwords
- One operand is a base-displacement address, the other is one byte in the instruction (that byte is the operand value):



SI instructions - MOVE IMMEDIATE

- The MOVE IMMEDIATE instruction copies one byte from the instruction to a storage address
 - Mnemonic **MVI**
 - Opcode **92**
- Example. To move the value 255 to displacement 8 from base register 7:

MVI 8(7) ,255

SI instructions - MOVE IMMEDIATE

- The instruction, in memory, will be:

MVI 8(7) ,255

0	8	12	16	20			
9	2	F	F	7	0	0	8

Operation code

Second operand
(immediate)

First operand (storage)

Assembler notation

- We'll see Assembler syntax in the next unit, but this is a good opportunity to introduce some Assembler notation
- Suppose you want to use MVI to move the character “A” to a location in storage
- Character “**A**” is (in EBCDIC), hex **C1**, or decimal 193. We know, from the previous example, that we can write this statement:

MVI 8(7) ,193

Assembler notation

- If you want to move the character “A” using MVI, you can write:

```
MVI    8(7),C'A'
```

- To move hex **C1** (which is the same value):

```
MVI    8(7),X'C1'
```

- Or even binary **11000001** (again, the same value as the two above):

```
MVI    8(7),B'11000001'
```

Assembler notation

- All these generate the same machine instruction:

MVI 8(7) , C'A'

MVI 8(7) , X'C1'

MVI 8(7) , B'11000001'

MVI 8(7) , 193

0		8		12		16		20	
9	2	C	1	7	0	0	8		

Assembler notation

- From now on, we'll use this notation to refer to values throughout the course:
- Instead of "hex FF", as we've been doing, we'll use **X'FF'**
- Same for binary: **B'1100'** instead of "binary 1100"
- Instead of "character Hello", **C'Hello'**
- Without prefix or quotes, the value is assumed to be decimal

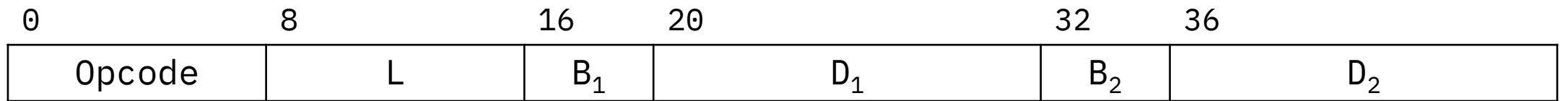
Instruction formats

- Next, SS instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register and another a storage address
Register-and-indexed-storage (RX)	
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address

SS instructions

- SS instructions occupy 3 halfwords
- In memory, they look like this:



- In addition to the two base-displacement operands, there is one byte for the first operand length

SS instructions - operand length

- To specify the operand length in Assembler, we use this format:

`displacement(length, base register)`

- In an instruction, the length field, **L**, is an 8-bit field, with a value from zero to 255 (X'FF')
- The maximum length is 256 bytes, when **L** is 255, or X'FF'
- The minimum is 1, when **L** is zero

SS Instructions - MOVE

- The SS MOVE instruction copies up to 256 bytes from the second operand to the first
- The number of bytes moved is determined by the 8-bit length field
 - From 1 byte (L=0) to 256 bytes (L=255, or X'FF')
- The mnemonic is **MVC**
- The opcode is **D2**

SS instructions - MOVE

- Example: move 48 bytes from the second operand to the first
 - The first operand is at displacement 16 from register 3
 - The second operand is at displacement 20 from register 4

MVC 16(48,3),20(4)

SS instructions - MOVE example

- The instruction, in memory, is:

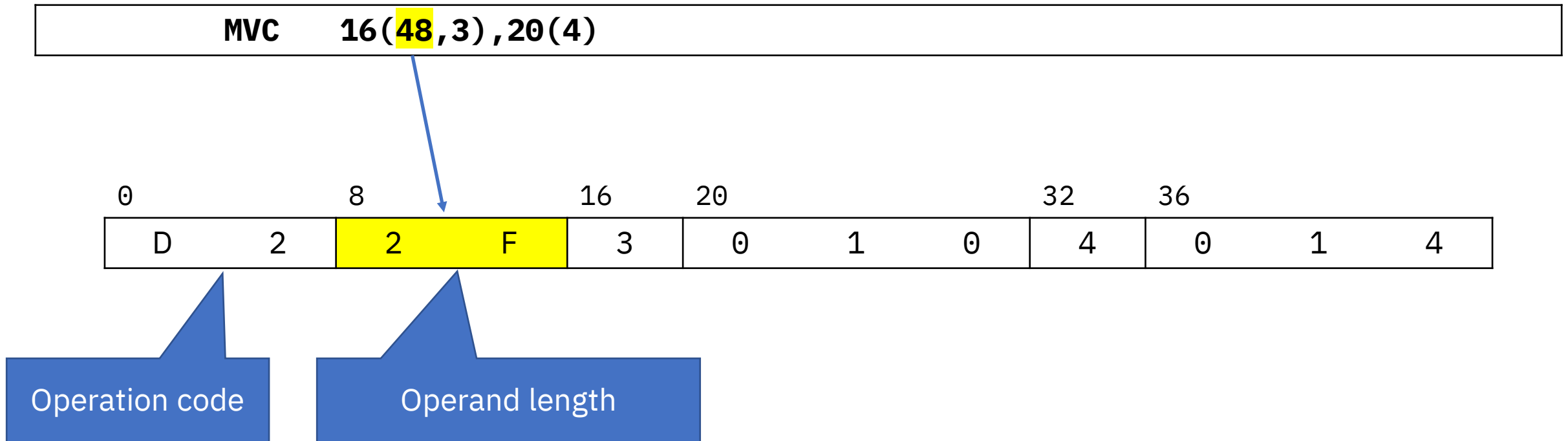
MVC 16(48,3),20(4)

0	8	16	20	32	36						
D	2	2	F	3	0	1	0	4	0	1	4

Operation code

SS instructions - MOVE example

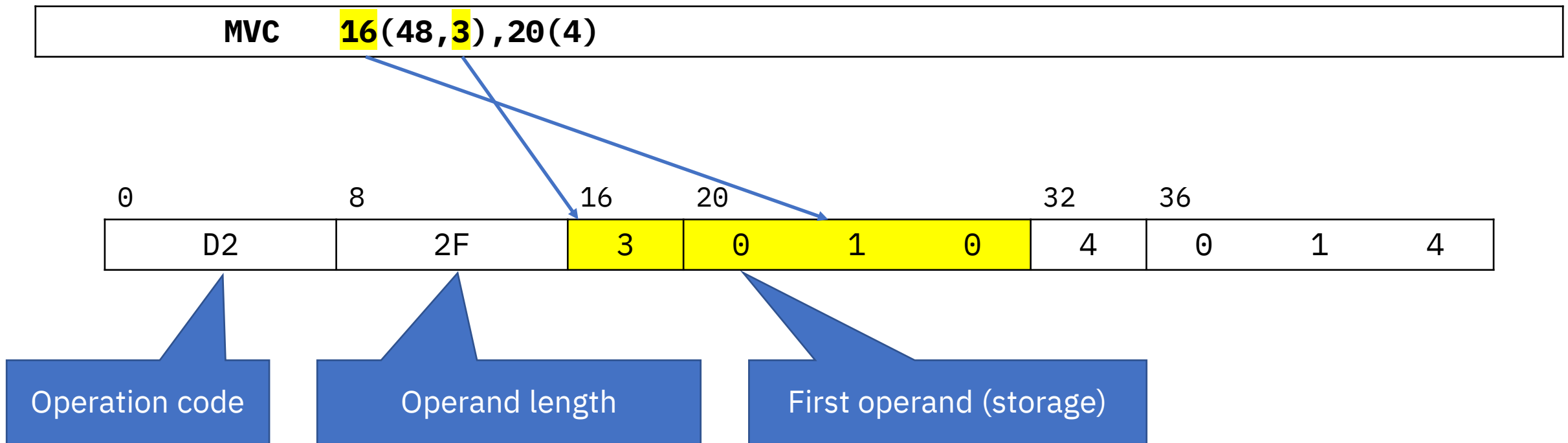
- The instruction, in memory, is:



- Note the length (hex 2F = 47): in the instruction, the encoded length is always *one less than the actual length*

SS instructions - MOVE example

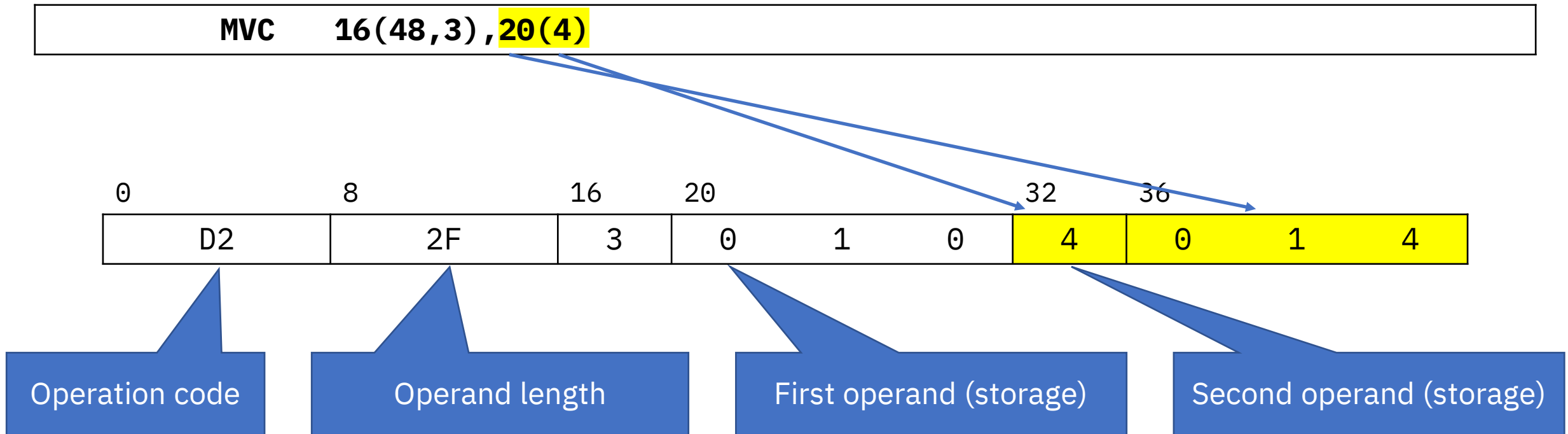
- The instruction, in memory, is:



- Note the length (hex 2F = 47): in the instruction, the length is always *one less than the actual length*

SS instructions - MOVE example

- The instruction, in memory, is:



- Note the length (hex 2F = 47): in the instruction, the length is always *one less than the actual length*

Instruction formats

- Next, RXY instructions

Format	Description
Register-and-register (RR)	Both operands are registers
Register-and-storage (RS)	One operand is a register and another a storage address
Register-and-indexed-storage (RX)	
Storage-and-immediate (SI)	One operand is a storage address and another an “immediate” value
Storage-and-storage (SS)	Both operands are storage addresses
Register-and-storage (RXY)	One operand is a register and another a long displacement address
Register-and-immediate (RI)	One operand is a register and another a 16-bit relative address